CBAP v1 参数手册

版本：1.0

整理日期：2026-10-24

用途：把 CBAP 线上协议、Kernel、安全策略、驱动清单、MCP 设备、Agent adapter、控制台的**实际参数和默认值**集中到一份文档

注意：本文以**当前实现**为准；`spec/cbap-v1.md` 是早期草案，文末列出实现与草案的差异

---

1. 版本与标识

参数	值
协议版本 `v`	`"1.0"`
协议名	CBAP / Cyberdyne Bridge Agent Protocol
传输	Unix domain socket（实现只支持 UDS）
分帧	UTF-8 NDJSON，一行一个 JSON object
单帧上限	`64 * 1024 = 65536` bytes
默认 Kernel socket	`<repo>/cbap.sock`
默认 Kernel id	`cyberdyne-legacy`
默认 Agent id	`cbap-client`
默认 Console Agent id	`cbap-console`
设备 ID 格式	`scheme://path`，如 `mc://bot/DeepSeek`
时间字段	`ts`：Unix epoch ms；`t_mono`：本机 monotonic ms
字符集	线上字段/`op`/错误码必须 ASCII；人类可读文本可本地化

参数	值

---

2. 传输与连接

2.1 Unix socket

项	值
地址	`AF_UNIX` / `SOCK_STREAM`
默认路径	`cbap.sock`（仓库根）
权限	bind 后 `0600`
对端校验	Linux `SO_PEERCRED`，默认只允许当前 uid
允许 uid	`KernelServer.allowed_uids` / CLI `--allowed-uid`（可重复）
连接上限	`max_connections` 默认 `128`（认证前）
认证超时	`hello_timeout` 默认 `2.0` 秒
会话上限	`max_sessions` 默认 `64`
读超时	socket 内部 `0.5` 秒轮询

2.2 TCP

草案允许 `127.0.0.1 + token`，当前实现**没有 TCP listener**。跨容器/Windows 需要另做 transport。

2.3 分帧

```
一行一个 JSON object
空行忽略
>64 KiB -> E_FRAME
解析失败 -> E_PROTOCOL
```

实现还会在缓冲区超过 `2 * MAX_FRAME` 且没有换行时清空并报 `E_FRAME`，连接保持。

---

3. 消息信封

所有消息：

字段	类型	必填	说明
`v`	string	是	固定 `"1.0"`
`type`	string	是	消息类型
`ts`	integer	是	Unix epoch ms
`id`	string	条件	request/result/ack/error/cancel 必填
`seq`	integer	条件	state/event
`op`	string	条件	request
`args`	object	条件	request
`ok`	boolean	条件	result
`error`	object	条件	失败
`payload`	object	否	载荷
`facts`	object	否	result 的独立事实
`deadline_ms`	integer	否	request 相对截止时间
`idempotency_key`	string	否	write 幂等键

未知字段忽略；未知 `type` 返回 `E_PROTOCOL`；未知 `op` 返回 `E_NOTSUP`。

3.1 `hello`

Agent → Kernel：

```
{
  "v": "1.0",
  "type": "hello",
  "id": "h-...",
  "ts": 1700000000000,
  "agent_id": "mindcraft-1",
  "role": "agent",
  "auth": {
    "token": "<agent-token>",
    "operator_token": "<optional>",
    "agent_token": "<optional>"
  },
  "encoding": "json",
  "capabilities": ["read", "write", "lease"],
  "limits": {"max_inflight": 4}
}
```

字段	说明
`agent_id`	1..128 字符（实现未强制上限；审计截断到 128）
`role`	草案为 `agent`；实现不使用它授权
`auth.token`	Kernel 全局 token；通常从 `<socket>.token` 自动读取
`auth.operator_token`	必须匹配 `<socket>.operator`，匹配后才授予 `estop`
`auth.agent_token`	若 policy 配置 `token_sha256`，必须提供；Kernel 比较 SHA-256
`capabilities`	**请求值**，不保证授予；实际以 `welcome.capabilities` 为准
`encoding`	实现只处理 JSON；CBOR 未实现
`limits`	当前忽略

3.2 `welcome`

```
{
  "v": "1.0",
  "type": "welcome",
  "id": "h-...",
  "ts": 1700000000000,
```

```
  "kernel_id": "cyberdyne-legacy",
  "protocol": "1.0",
  "session_id": "s-...",
  "capabilities": ["read", "write", "lease"],
  "limits": {
    "max_frame": 65536,
    "max_inflight": 4,
    "rate_hz": 10,
    "uid_rate_hz": 50,
    "max_sessions": 64
  },
  "devices": [
    {"device_id": "mc://bot/DeepSeek", "type": "minecraft-java", "state": "ready"}
  ]
}
```

`devices.state` 实现只产生 `ready` / `offline`。

3.3 `request`

```
{
  "v": "1.0",
  "type": "request",
  "id": "r-...",
  "ts": 1700000000000,
  "op": "write",
  "args": {"fd": "fd-1", "frame": {}},
  "deadline_ms": 5000,
  "idempotency_key": "optional"
}
```

`deadline_ms` 缺省/非法/≤0 时，write 的实现默认取 `5000`。

3.4 `ack`

```
{"v":"1.0","type":"ack","id":"r-...","ts":...,"state":"accepted"}
```

`state`：`accepted` 或 `running`。

`ack` 只表示受理；写入完成要等 `result`。

3.5 `progress`

草案有，当前 Kernel **没有发送** `progress`。保留。

3.6 `result`

```
{
  "v": "1.0",
  "type": "result",
  "id": "r-...",
  "ts": ...,
  "ok": true,
  "payload": {"fd": "fd-1"},
  "facts": {"block": "air"}
}
```

失败：

```
{
  "v": "1.0", "type": "result", "id": "r-...", "ts": ...,
  "ok": false,
  "error": {"code": "E_TIMEOUT", "message": "...", "retryable": true}
}
```

`ok:true` 不代表任务完成；完成由 `facts` + verify 判断。

3.7 `state`

```
{
  "v": "1.0",
  "type": "state",
  "seq": 102,
  "ts": ...,
  "t_mono": ...,
  "device_id": "mc://bot/DeepSeek",
  "payload": {}
}
```

`seq` 由 Kernel 全局递增；只有打开该设备 fd 的会话会收到。

`payload` 的实际通道见 § 6.1。

3.8 `event`

```
{
  "v": "1.0",
  "type": "event",
  "seq": 0,
  "ts": ...,
```

```
  "kind": "chat",
  "payload": {}
}
```

实现中 `seq` 目前固定为 `0`（`protocol.event` 默认值），不要依赖它判丢帧；状态用 `state.seq`。

3.9 `cancel`

```
{"v":"1.0","type":"cancel","id":"c-...","ts":...,"target":"r-..."}
```

当前实现：

```
只能取消本 session 的 pending
取消其他 session -> E_PERM
已完成的 target -> ok:true + already_done:true
```

3.10 `ping` / `pong`

实现支持：

```
{"v":"1.0","type":"ping","id":"p-1","ts":...}
{"v":"1.0","type":"pong","id":"p-1","ts":...}
```

草案未列，属于实现扩展。

3.11 `error`

```
{"v":"1.0","type":"error","id":"...","ts":...,"error":{...}}
```

错误对象见 §5。

---

4. CBAP-U 系统调用参数

4.1 `open`

```
{"op":"open","args":{"device_id":"mc://bot/DeepSeek","mode":"rw"}}
```

参数	类型	默认	说明
`device_id`	string	必填	必须匹配已注册 driver

参数	类型	默认	说明
`mode`	string	`rw`	`r` / `rw` / `exclusive`

返回：

```
{"fd":"fd-1","device_id":"mc://bot/DeepSeek"}
```

权限：

```
r          -> read
rw         -> read + write
exclusive  -> read + write + lease，并立即取租约
```

错误：`E_NOENT`、`E_PERM`、`E_BUSY`、`E_INVAL`。

4.2 `close`

```
{"op":"close","args":{"fd":"fd-1"}}
```

- 释放 fd 持有的租约；
- 取消该设备 pending；
- 若持有租约，调用 driver `estop()`；
- 返回 `{"closed":true}`。

4.3 `read`

```
{"op":"read","args":{"fd":"fd-1","channel":"body","timeout_ms":1000}}
```

参数	类型	默认	说明
`fd`	string	必填	会话内 fd
`channel`	string	无	缺省返回完整 frame
`timeout_ms`	integer	`0`	0 表示立即；无数据 `E_AGAIN`

返回：

```
{"frame": {"body": {...}, "state": {...}}}
```

`timeout_ms>0` 无数据 -> `E_TIMEOUT`；`=0` 无数据 -> `E_AGAIN`。

4.4 `write`

```
{
  "op":"write",
  "args":{
    "fd":"fd-1",
    "frame":{"channel":"body","op":"move","args":{"dir":[1,0],"ms":500}},
    "fencing_token": 3
  },
  "deadline_ms":5000,
  "idempotency_key":"move:1"
}
```

参数	类型	默认	说明
`fd`	string	必填	必须非只读
`frame`	object	必填	驱动方言
`fencing_token`	integer	无	可选；给了就必须匹配当前 lease
`deadline_ms`	integer	`5000`	请求级
`idempotency_key`	string	无	同 key 重试最多一次副作用

前置条件：

```
write capability
设备允许
非 safe mode
fd 非只读
持有有效 lease
inflight < max_inflight
rate <= rate_hz / uid_rate_hz
```

生命周期：

```
ack accepted
  -> driver.send
  -> result / event
```

错误前置条件在 ack 之前失败时，客户端若用 `CBAPClient.write()` 会等不到 ack，表现为客户端超时；这是当前 client 的一个已知行为。

4.5 `ioctl`

```
{"op":"ioctl","args":{"fd":"fd-1","request":"scan","args":{"radius":12,"step":2}}}
```

实现的 `request`：

request	args	说明
`scan`	`radius` 默认 12，`step` 默认 2	扫描环境
`watch`	`blocks` 数组	观察方块
`watch_items`	`items` 或 `list`	观察物品
`furnace_watch`	`p`，`每_ms` 默认 2000	熔炉观察

`ioctl` 实际要求 `write` capability，且受 rate/inflight 限制。

4.6 `poll`

```
{"op":"poll","args":{"timeout_ms":1000}}
```

当前实现只读取 `timeout_ms`，草案里的 `fds` / `events` 参数被忽略。

返回 session 事件队列：

```
{"events":[{"fd":"fd-1","kind":"readable"}]}
```

超时无事件返回 `ok:true` + 空数组。

4.7 `lease`

```
{"op":"lease","args":{"fd":"fd-1","ttl_ms":30000,"wait_ms":0}}
```

参数	类型	默认	说明
`fd`	string	必填	不能是 `mode=r`
`ttl_ms`	integer	Kernel `lease_ttl_ms`（30000）	最小 1ms
`wait_ms`	integer	`0`	>0 进入 FIFO 等待
`mode`	string	草案有	当前实现忽略

返回：

```
{
  "lease_id":"l-...",
  "fencing_token":3,
  "expires_at":1700000030000
}
```

错误：`E_BUSY`、`E_TIMEOUT`、`E_SAFETY`、`E_PERM`。

4.8 `release`

```
{"op":"release","args":{"fd":"fd-1"}}
```

返回：

```
{"released":true,"canceled":1}
```

`released:false` 表示当前 fd 没持有租约。释放会取消该设备的 pending。

4.9 `estop`

```
{"op":"estop","args":{"fd":"fd-1","reason":"operator"}}
```

参数	类型	默认	说明
`fd`	string/null	null	null 表示全局
`reason`	string	无	审计原因

前置能力：`estop`。普通 Agent 默认没有；必须使用 operator token。

返回：

```
{
  "safe_state":"off",
  "driver_ok":true,
  "canceled":1
}
```

行为：

```
取消 pending
清空 lease
```

```
设置 safe/safe_devices
调用 driver.estop
记录 audit
```

estop 锁存；实现没有 resume 路径（`resume` 永远 `E_PERM`）。

4.10 `stat`

```
{"op":"stat","args":{"fd":"fd-1"}}
```

返回：

```
{
  "device": {...},
  "health": {...},
  "lease": {"lease_id":"...","fencing_token":3,"agent_id":"...","expires_at":...},
  "safe": false,
  "security": {"degraded": false, "role": "agent"},
  "pending": 0,
  "fd": "fd-1"
}
```

4.11 `device_list`

```
{"op":"device_list","args":{}}
```

返回按 policy 过滤后的设备：

```
{"devices":[{"device_id":"...","type":"...","state":"ready"}]}
```

4.12 `resume`

```
{"op":"resume","args":{}}
```

当前实现固定：

```
{"ok":false,"error":{"code":"E_PERM","message":"resume requires admin (not implemented in v0)"}}
```

4.13 `cancel`（消息，不是 request）

见 §3.9；实现只允许取消本 session 的 pending。

---

5. 错误码

code	含义	可重试
`E_VERSION`	协议版本不匹配	否
`E_PROTOCOL`	帧/字段/类型错误	否
`E_AUTH`	未认证/token 错/agent token 错	否
`E_PERM`	权限/角色/设备不允许	否
`E_NOENT`	设备/fd/请求不存在	否
`E_BUSY`	租约冲突、pending、限流、inflight	是（退避）
`E_AGAIN`	暂无数据/背压	是
`E_TIMEOUT`	截止时间到	是（视副作用）
`E_INVAL`	参数非法	否
`E_NOTSUP`	不支持	否
`E_IO`	设备/传输错误	是
`E_CANCELED`	已取消	否
`E_SAFETY`	安全策略拒绝/safe mode	否
`E_STALE`	状态过期/探针未到	是
`E_FRAME`	帧过大/格式错误	否
`E_INTERNAL`	内核内部错误	是

错误对象：

```
{"code":"E_BUSY","message":"...","retryable":true,"detail":{}}
```

---

6. Kernel 状态与审计

6.1 `state.payload` 固定通道

`_state_payload()` 产生：

通道	内容
`body`	`pos`、`hp`、`food`、`on_ground`、`held`、`durability`、`acting`
`inventory`	`items`、`foods`、`weapons`
`world`	`scan`、`blocks`、`furnace`、`chat`
`players`	玩家
`near`	附近实体
`threats`	威胁
`drops`	掉落物
`stuck`	`count`、`ms`、`dir`
`attacked_by`	攻击者
`legacy_seq` / `legacy_t`	旧 hand 序号/时间
`state`	原始 driver state（通用设备）

6.2 审计行

```
{
  "seq": 12,
  "t": 1700000000000,
  "event": "write",
  "prev": "<sha256>",
  "hash": "<sha256>",
  "mac": "<optional hmac>",
  "...": "event fields"
}
```

事件名：

```
kernel_start, kernel_error, session_error, hello, auth_failure,
open, close, write, release, estop, watchdog_estop,
security_degrade, session_end
```

审计参数：

参数	默认
`audit_path`	`<repo>/logs/cbap-audit.jsonl`
文件权限	`0600`
`audit_mirror_path`	无
`audit_signing_key`	`CBAP_AUDIT_KEY` 或 secret backend
`checkpoint`	`<audit>.checkpoint`，字段 `seq/hash/mac`
`remote_url`	无；`CBAP_AUDIT_REMOTE_URL`
`remote_token`	无；`CBAP_AUDIT_REMOTE_TOKEN`
remote 队列	1000 行；满则丢并计数
remote 超时	3 秒；best-effort，无 ack 重试

`AuditChain.verify()` 检查 seq/prev/hash/mac/checkpoint；可检测尾部截断。

---

7. Kernel 参数

7.1 `KernelServer(...)` 构造参数

参数	类型	默认	说明
`driver`	object	无	单驱动
`drivers`	list	无	多驱动；与 `driver` 二选一

参数	类型	默认	说明
`socket_path`	str	`<repo>/cbap.sock`	UDS 路径
`kernel_id`	str	`cyberdyne-legacy`	welcome 中的 id
`token`	str/null	自动生成	全局 Agent token
`lease_ttl_ms`	int	`30000`	最小 1000
`audit_path`	str	`<repo>/logs/cbap-audit.jsonl`	审计
`policy`	str/dict/null	内置默认	能力策略
`allowed_uids`	set/list/null	当前 uid	peer uid 白名单
`max_sessions`	int	`64`	认证后会话
`max_connections`	int	`128`	认证前连接
`hello_timeout`	float	`2.0`	hello 超时秒
`max_inflight`	int	`4`	每 session
`rate_hz`	int	`10`	每 session 写/ioctl 每秒
`uid_rate_hz`	int	`50`	每 uid 每秒
`watchdog`	bool	`True`	启动状态看门狗
`audit_mirror_path`	str/null	无	本地镜像
`audit_signing_key`	str/null	`CBAP_AUDIT_KEY`	审计 HMAC
`audit_remote_url`	str/null	env	远程 NDJSON
`audit_remote_token`	str/null	env	Bearer token
`require_independent_facts`	bool	`False`	拒绝未标记 independent 的结果

参数	类型	默认	说明

7.2 Kernel CLI

```
python3 -m kernel.legacy_runtime [options]
```

选项	默认	对应
`--hand-socket`	`<repo>/cyberdyne.sock`	MC hand
`--cbap-socket`	`<repo>/cbap.sock`	Kernel UDS
`--device`	`<repo>/drivers/mc/device.json`	MC 描述
`--mapping`	`<repo>/drivers/mc/mapping.json`	MC 映射
`--token`	自动生成	Agent token
`--policy`	内置	policy JSON
`--allowed-uid`	当前 uid	可重复
`--max-sessions`	64	
`--max-connections`	128	
`--hello-timeout`	2.0	
`--max-inflight`	4	
`--rate-hz`	10	
`--uid-rate-hz`	50	
`--lease-ttl-ms`	30000	
`--audit`	`<repo>/logs/cbap-audit.jsonl`	
`--audit-mirror`	无	

选项	默认	对应
`--audit-remote-url`	无	
`--require-independent-facts`	false	
`--check`	—	连接 hand、打印 health 后退出

7.3 Token 文件

文件	内容	权限
`<socket>.token`	全局 Agent token	`0600`
`<socket>.operator`	operator token	`0600`
`<audit>.checkpoint`	`seq/hash/mac`	`0600`

- Kernel 启动时自动生成并写入；
- `CBAPClient` 自动读取 `.token`；
- operator token 需要显式读取，或控制台自动读取；
- 停止 Kernel 时删除 token 文件；
- 文件使用 `O_NOFOLLOW`，拒绝 symlink/非普通文件/owner 不匹配。

7.4 默认安全策略

```
{
  "default": {
    "devices": ["*"],
    "capabilities": ["read", "write", "lease"]
  },
  "agents": {}
}
```

含义：

```
普通 Agent：可读/写/租约，不能 estop
operator token：read/write/lease/estop
设备匹配：fnmatch
```

7.5 Policy 文件完整字段

```
{
  "default": {
    "devices": ["mc://bot/*"],
    "capabilities": ["read", "write", "lease"]
  },
  "agents": {
    "mindcraft-1": {
      "devices": ["mc://mind/Mindy"],
      "capabilities": ["read", "write", "lease"],
      "uids": [1000],
      "token_sha256": "<sha256 hex>"
    }
  }
}
```

字段	类型	说明
`default.devices`	string[]	fnmatch 模式
`default.capabilities`	string[]	`read/write/lease/estop`
`agents.<id>.devices`	string[]	覆盖默认
`agents.<id>.capabilities`	string[]	覆盖默认
`agents.<id>.uids`	int[]	为空不限制
`agents.<id>.token_sha256`	hex	为空不要求 agent token

`token_sha256` 对应 `hello.auth.agent_token` 的 SHA-256 hex。

operator 走独立 token，不受 policy 限制。

7.6 限流/异常

项	默认
认证失败窗口	60 秒
认证失败阈值	5 次 -> safe mode

项	默认
estop burst 窗口	60 秒
estop burst 阈值	10 次 -> safe mode
审计写失败	safe mode
watchdog 扫描	1 秒
watchdog 超时	`state_age_ms > 2 * safety.watchdog_ms`
写速率	10/s/session
uid 写速率	50/s/uid
最大 inflight	4/session

7.7 安全 secret backend

变量	值
`CBAP_SECRET_BACKEND`	`env` / `file` / `keyring`
`CBAP_SECRET_FILE`	0600 JSON 对象
`CBAP_KEYRING_SERVICE`	默认 `cbap`

配置的后端不可用时 **fail closed**，不会静默回退到 env。

---

8. 驱动 ABI 与描述符

8.1 CBAP-D 方法

方法	参数	返回
`describe()`	—	device.json
`health()`	—	`{ok, state_age_ms, ...}`

方法	参数	返回
`wait_state(timeout_s)`	秒	state dict / None
`send(command)`	dict	bool
`map_action(frame)`	frame	`{cmd, ...}`
`estop()`	—	bool
callbacks	`on_state/on_result/on_event/on_disconnect`	—

当前 Kernel 的 driver 是**进程内对象**；草案的独立进程 CBAP-D 尚未实现。

8.2 `device.json` 字段

字段	必填	说明
`device_id`	是	唯一设备 ID
`type`	是	设备类型
`cbap`	是	`"1.0"`
`display_name`	否	展示名
`channels`	是	通道读/写/ioctl
`capabilities`	是	能力
`limits`	否	`max_frame`/`max_inflight`/`rate_hz`/`max_command_ms`
`safety.estop`	是	非空；Kernel 注册检查
`safety.watchdog_ms`	是	int；Kernel 注册检查
`safety.safe_state`	否	安全状态
`schemas`	否	payload schema URI

字段	必填	说明
`metadata`	否	驱动元数据

8.3 `mapping.json` 字段

字段	说明
`cbap`	`1.0`
`device_id`	匹配
`ops`	动作名 -> `{channel, op, args, legacy_args?}`
`verify`	动作 -> 事实判据

`args` 类型标记：

```
vec2 / vec3 / int / bool / string / entity / object
前缀 ? 表示可选
```

8.4 MC 设备参数

`drivers/mc/device.json`：

```
device_id: mc://bot/DeepSeek
type: minecraft-java
max_frame: 65536
max_inflight: 1
rate_hz: 5
max_command_ms: 120000
safety.estop: stop+idle
safety.watchdog_ms: 5000
safety.safe_state: idle
```

MC 映射 21 个 op：

op	channel	args
`move`	body	`dir:vec2, ms:int, ?sprint:bool, ?delay_ms:int`

op	channel	args
`jump`	body	`?count:int`
`jumpattack`	body	`target:entity`
`sneak`	body	`?ms:int`
`swimdown`	body	`?ms:int`
`eat`	body	无
`attack`	body	`target:entity, ?ms:int`
`goto`	body	`p:vec3`
`stop`	body	无
`dig`	world	`p:vec3, ?delay_ms:int`
`place`	world	`p:vec3, ?item:string`
`use`	world	`p:vec3`
`collect`	world	`item:string, block:string, count:int`
`chest_take`	world	`p:vec3, item:string, count:int`
`chest_put`	world	`p:vec3, item:string, count:int, ?keep:int`
`chest_audit`	world	`p:vec3`
`furnace_put`	world	`p:vec3, input:object, fuel:object`
`furnace_take`	world	`p:vec3, item:string, count:int`
`give`	world	`item:string, count:int, ?to:string`
`craft`	inventory	`item:string, ?count:int`

op	channel	args

`say` chat `text:string`

verify：

```
{
  "dig": {"type":"block","p":"$args.p","equals":"air"},
  "place": {"type":"block","p":"$args.p","equals":"$args.item"}
}
```

---

9. MCP Device Driver 参数

`MCPDeviceDriver(manifest_path, connect_timeout=30.0)`

9.1 Manifest 完整字段

```
{
  "device_id": "mcp://lab/light-01",
  "display_name": "Lab light",
  "transport": {
    "type": "stdio",
    "command": ["python3", "server.py"],
    "cwd": ".",
    "env": {},
    "inherit_env": false,
    "env_allowlist": ["PATH", "HOME"],
    "rlimits": {"RLIMIT_NOFILE": 256},
    "sandbox": {"type": "bwrap", "writable_cwd": false}
  },
  "tools": {
    "turn_on": {"tool": "turn_on", "args": {}},
    "turn_off": {"tool": "turn_off", "args": {}}
  },
  "facts": {
    "source": "tool",
    "tool": "get_state",
    "independent": false,
```

```
    "args": {},
    "uri": "",
    "command": [],
    "cwd": ".",
    "env": {},
    "env_allowlist": []
  },
  "estop": {
    "tool": "turn_off",
    "args": {},
    "expect": {"on": false},
    "tolerance": 0.0,
    "hardware_command": []
  },
  "watchdog_ms": 10000,
  "poll_interval": 0.2,
  "facts_timeout_ms": 2000,
  "pin_tools": {
    "tools_sha256": "<hex>",
    "instructions_sha256": "<hex>"
  },
  "signature": "<hmac-sha256 hex>"
}
```

9.2 字段/默认值

字段	默认	说明
`device_id`	必填	设备 ID
`display_name`	device_id	展示名
`transport.type`	必填	只支持 `stdio`
`transport.command`	必填	argv 数组，不走 shell
`transport.cwd`	manifest 目录	相对 manifest 目录
`transport.env`	`{}`	显式环境变量
`transport.inherit_env`	`false`	是否继承完整环境

字段	默认	说明
`transport.env_allowlist`	`PATH/HOME/LANG/LC_ALL/LC_CTYPE/TMPDIR`	inherit=false 时允许
`transport.rlimits`	无	传给 `resource.setrlimit`
`transport.sandbox`	无	`bwrap` / `nsjail`；未安装失败
`tools`	必填	CBAP op -> MCP tool
`facts.source`	`tool`	`tool` / `resource` / `command`
`facts.tool`	无	source=tool
`facts.uri`	无	source=resource
`facts.command`	无	source=command，argv
`facts.independent`	`false`	声明独立事实源
`estop.tool`	无	MCP 急停工具
`estop.expect` / `verify`	无	安全状态事实判据
`estop.tolerance`	无	数值容差
`estop.hardware_command`	无	本地继电器 argv；失败则 estop 失败
`watchdog_ms`	`10000`	Kernel 看门狗
`poll_interval`	`0.2` 秒，最小 0.05	状态轮询
`facts_timeout_ms`	`2000`	事实稳定窗口
`pin_tools`	无	工具/instructions 摘要
`signature`	无	manifest HMAC

9.3 工具名

Manifest `tools` 是**白名单**：

```
CBAP op -> MCP tool
```

MCP server 广告的 tools 会记录到 `health.advertised_tools`；未在 manifest 中的工具不会被 CBAP 暴露。

9.4 验证状态

状态	含义
`verified`	tool 成功 + 事实匹配
`mismatch`	tool 成功 + 事实不匹配
`observed`	只读取事实，没有 expect
`unverifiable`	没有 facts 源
`error`	tool/传输失败

`MCPDeviceDriver.send(command)`：

```
{"op":"turn_on","args":{},"expect":{"on":true},"tolerance":0.1}
```

返回 bool；详细信息在 `last_verification()` / `health()`。

9.5 MCP 子进程

项	值
transport	stdio JSON-RPC 2.0
初始化协议	`2024-11-05`
单行上限	8 MiB
stderr 单行	4 KiB，保留 80 行
默认超时	`connect_timeout=30s`
进程组	`start_new_session=True`；close/estop 杀进程组
环境	默认不继承完整环境

项	值
manifest 签名	`CBAP_MANIFEST_KEY` / secret backend
强制签名	`CBAP_REQUIRE_SIGNED_MANIFEST=1`
facts 输出	最大 1 MiB

9.6 MCP 探针 CLI

```
python3 tools/mcp_device_probe.py manifest.json \
  --op turn_on \
  --args '{}' \
  --expect '{"on":true}' \
  --tolerance 0.1 \
  --json
```

退出码：

```
0 verified / observed
2 mismatch
3 unverifiable
4 transport/driver error
```

---

10. Agent Adapter 参数

10.1 MCP Mineflayer Agent

`agents/mcp-mineflayer/driver/driver.py`

`MCPDriver(...)`：

参数	默认	说明
`server_path`	`CBAP_MCP_SERVER`	mineflayer-mcp 路径
`host`	`127.0.0.1`	MC 服务器

参数	默认	说明
`port`	`25565`	MC 端口
`username`	`MCPBody`	bot 名
`version`	`1.21.1`	MC 版本
`node_bin`	`CBAP_NODE_BIN` / `node`	Node
`lock_path`	`/tmp/cbap-mcp-<username>.lock`	单实例锁
`command`	自动构造	覆盖 argv
`cwd`	server_path	工作目录
`device_path`	`driver/device.json`	描述符
`mapping_path`	`driver/mapping.json`	映射
`device_id`	`mcp://mc/MCPBody`	覆盖设备 ID
`poll_interval`	0.5，最小 0.2	轮询
`connect_timeout`	40.0	连接
`auto_restart`	true	MCP 崩溃重启
`restart_delay`	1.0，最小 0.2	重启间隔

设备限制：

```
max_frame: 65536
max_inflight: 2
rate_hz: 2
max_command_ms: 120000
safety.watchdog_ms: 10000
safe_state: offline
```

MCP 映射：

op	channel	args
`move`	body	`dir:vec2, ?ms:int, ?p:vec3`
`goto`	body	`p:vec3`
`dig`	world	`p:vec3`
`place`	world	`p:vec3, ?item:string`
`attack`	world	`target:int`
`use`	world	`?p:vec3`
`craft`	inventory	`item:string, ?count:int`
`say`	chat	`text:string`
`equip`	inventory	`name:string, ?destination:string`

Dashboard：

```
--server-path, --node, --mc-host, --mc-port, --mc-version,
--username, --device-id, --lock, --http-port (8099)
```

10.2 Mindcraft

Adapter `agents/mindcraft/adapter/adapter.mjs`：

参数	默认
`--port`	`8080`
`--agent`	`Mindy`
`--socket`	`/tmp/mindcraft-cbap.sock`
`--moduleRoot`	本地 mindcraft package.json

设备：

```
device_id: mc://mind/Mindy
```

```
max_frame: 65536
max_inflight: 1
rate_hz: 1
max_command_ms: 30000
safety.watchdog_ms: 5000
safe_state: agent-stopped
```

映射：

op	channel	args
`prompt`	chat	`text:string`
`stop`	body	无

10.3 Voyager

Adapter `agents/voyager/adapter/adapter.mjs`：

参数	默认
`--bridge`	`http://127.0.0.1:3000`
`--socket`	`/tmp/voyager-cbap.sock`
`--port`	`25565`

设备：

```
device_id: mc://voyager/bot
max_frame: 65536
max_inflight: 1
rate_hz: 1
max_command_ms: 60000
safety.watchdog_ms: 5000
safe_state: bridge-stopped
```

映射：

op	channel	args
`forward`	body	无

op	channel	args
`exec`	body	`code:string`
`say`	chat	`text:string`
`stop`	body	无

10.4 Cyberdyne / Alice

- Cyberdyne 自己脑/手使用 `McDriver` + `drivers/mc/device.json`。
- Alice 只有源码归档，不是活跃子仓，不能声明可运行。

---

11. Operator Console 参数

11.1 CLI

```
python3 console/server.py \
  --socket ./cbap.sock \
  --audit ./logs/cbap-audit.jsonl \
  --port 8765 \
  --token '<kernel-agent-token>' \
  --operator-token '<operator-login-token>'
```

参数	默认	说明
`--socket`	`CBAP_SOCKET` 或 `<repo>/cbap.sock`	Kernel
`--audit`	`CBAP_AUDIT` 或 `<repo>/logs/cbap-audit.jsonl`	审计
`--port`	`8765`	只绑 `127.0.0.1`
`--token`	自动读 `<socket>.token`	Kernel agent token
`--operator-token`	`CBAP_CONSOLE_OPERATOR_TOKEN` 或自动生成	登录 token，打印到终端

11.2 网络/认证

项	值
bind	`127.0.0.1` only
Host 允许	`127.0.0.1:<port>` / `localhost:<port>`
Origin 允许	同 Host 的 http
登录限速	5 次/60s/IP
session TTL	8 小时
Cookie	`cbap_console_session`；HttpOnly；SameSite=Strict；Path=/
CSP	self only；frame-ancestors none
CORS	无
页面 token	无；登录后 cookie

11.3 HTTP API

方法	路径	认证	说明
GET	`/`	否	控制台 HTML
GET	`/static/app.js`	否	JS
GET	`/static/style.css`	否	CSS
GET	`/api/session`	否	`{authenticated:bool}`
POST	`/api/login`	operator token	设置 cookie
POST	`/api/logout`	否	清 session
GET	`/api/status`	cookie	设备/租约/facts
GET	`/api/audit?limit=80`	cookie	审计，limit 1..500

方法	路径	认证	说明
POST	`/api/estop`	cookie	`{fd:string|null,reason:string}`

`/api/status` 返回：

```
{
  "ok": true,
  "ts": 1700000000.0,
  "kernel": {"socket": "...", "connected": true},
  "devices": [{
    "device_id": "...",
    "type": "...",
    "online": true,
    "fd": "fd-1",
    "facts": {},
    "facts_error": null,
    "safe": false,
    "lease": null,
    "pending": 0,
    "health": {},
    "security": {"degraded": false, "role": "operator"},
    "channels": ["body", "state"]
  }]
}
```

`/api/estop` 返回：

```
{
  "ok": true,
  "result": {"safe_state":"off","driver_ok":true,"canceled":1},
  "status": {"ok":true,"devices":[]}
}
```

11.4 Console 后端行为

```
每个设备 open(mode="r")
缓存 fd
每次刷新 stat + read(timeout_ms=0)
不获取 lease
不直连 driver
```

```
estop 走 Kernel
没有 resume endpoint
```

---

12. 环境变量总表

变量	默认	作用
`CBAP_SOCKET`	`<repo>/cbap.sock`	Console 默认 Kernel socket
`CBAP_AUDIT`	`<repo>/logs/cbap-audit.jsonl`	Console 默认审计
`CBAP_CONSOLE_OPERATOR_TOKEN`	自动生成	Console 登录 token
`CBAP_AUDIT_KEY`	无	审计 HMAC
`CBAP_AUDIT_REMOTE_URL`	无	远程审计 URL
`CBAP_AUDIT_REMOTE_TOKEN`	无	远程审计 Bearer
`CBAP_MANIFEST_KEY`	无	MCP manifest HMAC
`CBAP_REQUIRE_SIGNED_MANIFEST`	`0`	`1` 拒绝未签名 manifest
`CBAP_SECRET_BACKEND`	`env`	`env/file/keyring`
`CBAP_SECRET_FILE`	无	file backend JSON
`CBAP_KEYRING_SERVICE`	`cbap`	keyring service
`CBAP_MCP_SERVER`	空	MCP agent server path
`CBAP_NODE_BIN`	`node`	MCP agent Node
`MC_RCON_PASSWORD`	无	MC 工具，不属于 CBAP 协议

---

13. 关键默认值汇总

```
MAX_FRAME                65536 bytes
Kernel lease TTL         30000 ms
Kernel max_sessions      64
Kernel max_connections   128
Kernel hello_timeout     2.0 s
Kernel max_inflight      4
Kernel rate_hz           10/s/session
Kernel uid_rate_hz       50/s/uid
Auth failure threshold   5/60s
Estop burst threshold    10/60s
Watchdog scan            1 s
MCP poll_interval        0.2 s
MCP facts_timeout_ms     2000 ms
MCP watchdog_ms          10000 ms
MCP stdout line          8 MiB
MCP stderr line          4 KiB
MCP facts output         1 MiB
Console port             8765
Console session TTL      8 h
Console login limit      5/60s/IP
Console audit limit      80 default / 500 max
```

---

14. 实现与草案的差异

这些差异是实际行为，不要按草案误用：

项	spec/cbap-v1.md	当前实现
TCP	可选支持	未实现，只有 UDS
CBOR	hello 可协商	未实现
`poll`	`fds`/`events`	只读 `timeout_ms`，返回 session 事件
`lease.mode`	有	忽略
`progress`	有	不发送

项	spec/cbap-v1.md	当前实现
`event.seq`	单调递增	固定 `0`
`estop` 权限	MUST 总是允许	必须 operator token；普通 Agent `E_PERM`
`cancel` 他人	admin 可取消	当前 `E_PERM`
`resume`	admin 可恢复	未实现，固定 `E_PERM`
CBAP-D 独立进程	支持	只支持进程内 driver
hello token	UDS MAY	Kernel 生成 token，实际要求
hello capabilities	请求权限	以 policy 授予为准
global state	推送	只推给持有该设备 fd 的会话
`write` 前置失败	result	无 ack；`CBAPClient.write` 可能表现为超时
spec 用例数	20 条	当前 89 条实现测试

---

15. 命令与验证

```
# 全量 CBAP
python3 tools/cbap_conformance.py --target fake

# 控制台测试
python3 -m unittest -v tests.cbap.test_console

# 安全测试
python3 -m unittest -v tests.cbap.test_security

# MCP 设备测试
python3 -m unittest -v tests.cbap.test_mcp_device

# MCP 单设备探针
python3 tools/mcp_device_probe.py drivers/mcp_device/examples/light.json \
  --op turn_on --expect '{"on":true}'
```

当前状态：

```
CBAP conformance: 89/89
console tests:    3
security tests:   14
MCP device tests: 11
```

---

16. 相关文件

```
spec/cbap-v1.md
spec/schema/message.schema.json
spec/schema/device.schema.json
spec/schema/mapping.schema.json
kernel/legacy_runtime/protocol.py
kernel/legacy_runtime/server.py
kernel/legacy_runtime/security.py
kernel/legacy_runtime/secrets.py
kernel/legacy_runtime/client.py
drivers/mc/device.json
drivers/mc/mapping.json
drivers/mcp_device/driver.py
drivers/mcp_device/README.md
drivers/mcp_device/sandbox.py
console/server.py
console/backend.py
console/README.md
docs/SECURITY-HARDENING.md
```
